Documento de Divisão de Funções Técnicas – Inventra

1. IDENTIFICAÇÃO

Projeto	Inventra
Documento	Divisão de Funções Técnicas
Versão do documento	2.1
Data	27/10/2026
Autor	Equipe Inventra 2°
Status	Combinado interno do grupo

Histórico de versões

Versão	Data	Alterações
1.0	06/07/2026	Divisão de funções por matéria, resumo por integrante e regras de combinação do grupo.
2.1	27/10/2026	Nova seção 6, Fluxo de trabalho: tarefas no Jira, fluxo de branches, padrão de commits, pull requests e code review, integração contínua, reuniões e resolução de conflitos de código. Versão alinhada ao Documento de Requisitos Funcionais 2.1.

2. ESCOPO DO DOCUMENTO

2.1 Descrição geral

Este documento registra a divisão de responsabilidades técnicas entre os integrantes do grupo para o desenvolvimento do Inventra, sistema de gestão de estoque e controle de movimentações em cozinhas institucionais ou comerciais. A divisão é organizada por matéria do curso, associando cada uma às frentes técnicas do sistema. A partir da versão 2.1, o documento também descreve como o trabalho flui entre os integrantes: como as tarefas são distribuídas e acompanhadas, como o código é versionado, revisado e integrado, e como o grupo se comunica.

3. DIVISÃO DE FUNÇÕES POR MATÉRIA

Código	Matéria	Integrantes responsáveis
DF01	Banco de Dados 2	Felipe Kogake, Eduardo Passos
DF02	Engenharia de Software	Felipe Kogake, Davi Arakaki
DF03	Desenvolvimento 2	Felipe Kogake, Davi Arakaki
DF04	Mobile	João Maldonado, Felipe Kogake
DF05	Desenvolvimento de Aplicações Dinâmicas	João Maldonado, João Prado, Rafael Passos
DF06	Desenvolvimento de Operações Ágeis	Eduardo Passos, Davi Arakaki

Código	Matéria	Integrantes responsáveis
DF07	UX	Eduardo Passos, João Prado, Rafael Passos
DF08	Modelagem de Dados	João Maldonado, Davi Arakaki
DF09	Business Intelligence	Rafael Passos, João Prado
DF10	Inteligência Artificial	Rafael Passos, João Maldonado

4. RESUMO POR INTEGRANTE

Integrante	Matérias / frentes técnicas
Felipe Kogake	Banco de Dados 2, Engenharia de Software, Desenvolvimento 2, Mobile
Eduardo Passos	Banco de Dados 2, Desenvolvimento de Operações Ágeis, UX
Davi Arakaki	Engenharia de Software, Desenvolvimento 2, Desenvolvimento de Operações Ágeis, Modelagem de Dados
João Maldonado	Mobile, Modelagem de Dados, Inteligência Artificial
João Prado	Desenvolvimento de Aplicações Dinâmicas, Business Intelligence, UX
Rafael Passos	Desenvolvimento de Aplicações Dinâmicas, UX, Business Intelligence, Inteligência Artificial

5. REGRAS DE COMBINAÇÃO DO GRUPO

- **Comunicação centralizada:** avisos, dúvidas e decisões do projeto são registrados no grupo oficial (WhatsApp), evitando combinados feitos só entre duas pessoas.
- **Avisar se travar:** quem ficar travado em uma tarefa por mais de um dia avisa o grupo, para que outra pessoa da mesma frente possa ajudar.
- **Revisão cruzada antes de subir código:** nenhuma alteração relevante entra na branch principal sem revisão de pelo menos um outro integrante da mesma matéria (detalhado na seção 6.4).
- **Padrão de versionamento:** branches separadas por funcionalidade e mensagens de commit no padrão definido (seções 6.2 e 6.3).
- **Prazos internos:** quem não conseguir cumprir um prazo combinado avisa com antecedência, para que as tarefas sejam reorganizadas.
- **Reuniões sob demanda:** o grupo se reúne quando há necessidade (seção 6.6).
- **Resolução de conflitos:** divergências técnicas são discutidas no grupo com argumentos, buscando consenso; sem acordo, a decisão é tomada por votação.
- **Backup e documentação:** cada parte do sistema mantém documentação mínima (como funciona e como rodar).

6. FLUXO DE TRABALHO

6.1 Tarefas no Jira

O trabalho é acompanhado no Jira, no projeto **INV2** (inventraoficial.atlassian.net). A organização segue três níveis:

- **Épico por matéria:** cada matéria do curso tem um épico (por exemplo, Banco de Dados 2, Desenvolvimento de Aplicativos Móveis, Engenharia e Qualidade de Software), com os entregáveis exigidos.
- **Tarefas filhas:** cada entregável vira uma tarefa dentro do épico, com descrição, critérios de aceite, prioridade e categoria (OBRIGATÓRIO ou EXTRA).
- **Responsável:** a tarefa é atribuída a um dos integrantes responsáveis pela matéria (seção 3). Quem pega a tarefa move o card no quadro: Backlog → Em andamento → Concluído.

O quadro de épicos dá a visão geral de cada matéria, e o quadro Kanban mostra o andamento das tarefas. Documentos de entrega, como PDFs e diagramas, são anexados na tarefa correspondente.

6.2 Fluxo de branches

A branch **main** guarda a versão estável de cada repositório. Ninguém faz commit direto nela: toda mudança nasce em uma branch própria e entra por pull request (seção 6.4). O nome da branch segue o padrão `<tipo>/objetivo`, com o mesmo vocabulário de tipos dos commits:

```
main
 ├── feat/tela-login          nova funcionalidade
 ├── fix/autenticacao-jwt     correção de bug
 ├── docs/design-patterns     documentação
 └── chore/kover              configuração e manutenção
```

- **Uma branch por funcionalidade:** cada tarefa do Jira costuma virar uma branch, e o nome da branch ou do PR cita a tarefa.
- **Branches empilhadas:** quando uma funcionalidade depende de outra que ainda está em revisão, a nova branch sai da anterior, e o PR aponta para ela em vez da main. Depois que a anterior é mergeada, a seguinte passa a apontar para a main.
- **Branch atualizada antes do merge:** antes de mergear, a branch recebe as últimas mudanças da main (seção 6.7).

6.3 Padrão de commits

As mensagens de commit seguem o formato `<tipo>(escopo): descrição curta`, em português:

Tipo	Quando usar	Exemplo
feat	Nova funcionalidade	`feat(login): adiciona autenticação com token JWT`
fix	Correção de bug	`fix(desconto): corrige cálculo no carrinho`
chore	Configuração, dependências, build	`chore(dependencias): atualiza dependências`
refactor	Melhoria de código sem mudar comportamento	`refactor(navegacao): extrai funções de salvar`
style	Mudança só visual	`style(botao): ajusta cor do botão`
docs	Documentação	`docs(readme): explica como rodar o app`
test	Testes	`test(estoque): cobre baixa sem saldo`
revert	Desfazer um commit anterior	`revert: desfaz ajuste de layout`

6.4 Pull requests e code review

- **Template padrão:** todo PR usa o template da organização no GitHub (InventraTech), com as seções Descrição, Tipo de mudança, Como testar, Checklist, Evidências e Observações para o

revisor.

- **Ligação com o Jira:** a descrição do PR inclui o link da tarefa INV2 correspondente.
- **Revisão cruzada:** pelo menos um outro integrante da mesma matéria revisa o PR antes do merge. Comentários são resolvidos na própria branch, com novos commits.
- **Critério de merge:** o PR só é mergeado com a revisão aprovada e a integração contínua passando (seção 6.5).
- **Histórico preservado:** quando o histórico de commits é parte da entrega (por exemplo, o ciclo do TDD), o merge é feito sem squash, para manter os commits.

6.5 Integração contínua (CI)

A integração contínua roda no GitHub Actions, com workflows reutilizáveis guardados na organização InventraTech. No repositório mobile, todo PR e todo push para a main disparam o workflow de Kotlin, que roda três verificações:

Verificação	Ferramenta	O que garante
Estilo de código	ktlint	Formatação padronizada do Kotlin.
Análise estática	detekt	Código sem problemas comuns de complexidade e manutenção.
Testes	Testes unitários (JUnit)	Lógica de ViewModels, repositórios e navegação funcionando.

Se alguma verificação falhar, o PR não é mergeado até a correção. A cobertura de testes é medida com o Kover, como apoio ao requisito não funcional de testabilidade.

6.6 Reuniões e comunicação

- **Reuniões sob demanda:** o grupo não tem reunião fixa. Uma reunião, presencial ou online, é marcada quando há bloqueio em uma tarefa, entrega próxima ou dependência entre frentes (por exemplo, um endpoint do backend necessário para o app).
- **Convocação pelo grupo oficial:** a reunião é combinada no WhatsApp do grupo, e as decisões tomadas são registradas lá ou na tarefa do Jira.
- **Acompanhamento contínuo:** entre as reuniões, o andamento é acompanhado pelo quadro do Jira e pelos PRs abertos.

6.7 Resolução de conflitos de código

- **Atualizar antes do merge:** quem abriu o PR traz as mudanças da main para a sua branch antes de mergear.
- **Quem resolve:** os conflitos são resolvidos por quem abriu o PR. Se o conflito envolve código de outra pessoa, ela é chamada para decidir junto.
- **Verificar de novo:** depois de resolver, a integração contínua roda outra vez e o PR volta para revisão se a resolução mudou a lógica.